LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3
JAVA BASICS II

Disusun Oleh:

Glory Hardy Febriando Sianturi (105223013)

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Program ini merupakan prototipe sistem Kiosk (Mesin Tiket Mandiri) untuk acara "Universitas Pertamina SoundFest 2026". Karena sistem ini beroperasi tanpa pengawasan staf, program dirancang agar kebal terhadap *error* dari input pengguna yang tidak masuk akal (seperti usia minus atau ratusan tahun), mampu mencegah *overselling* (kelebihan penjualan), dan dapat memproses pembelian tiket sekaligus melakukan validasi usia untuk setiap tiket secara berurutan.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	stockVIP, stockFestival, stockTribune	int	Menyimpan jumlah sisa kuota masing-masing kategori tiket yang bisa dibeli.
2	hargaVIP, hargaFestival, hargaTribune	int	Menyimpan nilai harga tetap (konstanta) dari masing-masing kategori tiket.
3	minUsiaVIP, minUsiaFestival, minUsiaTribune	int	Menyimpan batas syarat minimal usia untuk masing-masing kategori tiket.
4	pilihan	int	Menyimpan data input angka dari pengguna saat memilih menu di halaman utama.
5	jumlahTiket	int	Menyimpan data input jumlah tiket yang ingin dipesan pengguna dalam satu transaksi..
6	stokSaatIni, hargaSaatIni, minUsiaSaatIni	int	Variabel penampung sementara (*helper*) untuk menyimpan atribut kategori tiket yang sedang dipilih agar pengecekan lebih dinamis.
7	namaKategori	String	Menyimpan teks nama kategori (contoh: "VIP") yang dipilih untuk mempermudah cetak pesan di layar.
8	tiketBerhasil	int	Menghitung dan menyimpan jumlah tiket yang sukses lolos verifikasi usia untuk dicetak di nota.
9	totalTagihan	int	Menyimpan akumulasi total harga dari tiket-tiket yang sukses divalidasi.
10	batalDarurat	boolean	Menjadi penanda (*flag*) status apakah pengguna menekan angka -

			1 untuk membatalkan proses pengisian usia.
11	usia	int	Menyimpan data input usia pelanggan untuk divalidasi satu per satu pada setiap tiket.
12	i	int	Sebagai *counter* atau variabel iterasi di dalam perulangan for untuk melacak urutan tiket (Tiket ke-1, ke-2, dst).

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	Scanner(System.in)	*Constructor*	Membuat objek baru dari class Scanner untuk menerima input dari keyboard.
2	main(String[] args)	*Procedural*	Method utama yang menjalankan urutan instruksi program dari awal sampai akhir.
3	nextInt()	*Functional*	Membaca input angka bulat (integer) dari pengguna dan mengembalikan nilai integer tersebut.
4	print()	*Procedural*	Menampilkan teks ke layar tanpa menambahkan baris baru (kursor tetap di sebelah teks).
5	println()	*Procedural*	Menampilkan teks atau hasil proses ke layar, lalu secara otomatis memindahkan kursor ke baris baru.
6	close()	*Procedural*	Menutup objek Scanner setelah selesai digunakan untuk membebaskan sumber daya memori.

IV. Dokumentasi dan Pembahasan Code

4.1. Inisialisasi dan Deklarasi Variabel

Pada bagian ini, program menyiapkan objek Scanner untuk menangkap input dari kasir.

```java
import java.util.Scanner;

Windsurf: Refactor | Explain
public class KioskSoundFest {
    Run | Debug | Windsurf: Refactor | Explain | Generate Javadoc | ×
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int stockVIP = 5;
        int stockFestival = 25;
        int stockTribune = 35;

        final int hargaVIP = 1500000;
        final int hargaFestival = 800000;
        final int hargaTribune = 500000;

        final int minUsiaVIP = 18;
        final int minUsiaFestival = 15;
        final int minUsiaTribune = 0;
```

Pembahasan:

Pada awal program, sistem menggunakan class Scanner untuk menerima input dari keyboard. Selanjutnya, dideklarasikan variabel-variabel fundamental seperti stok tiket, konstanta harga, dan konstanta batas minimum usia. Penggunaan tipe data int mendominasi karena sistem ini murni hanya memproses bilangan bulat, tanpa desimal atau teks panjang.

4.2. Perulangan Utama (Menu Kiosk) & Input Pilihan

```java
while (true) {
    System.out.println("\n==========================================");
    System.out.println("  KIOSK TIKET UNIVERSITAS PERTAMINA SOUNDFEST 2026  ");
    System.out.println("==========================================");
    System.out.println("Sisa Stok Tiket: VIP (" + stockVIP + "), Festival (" + stockFestival + "), Tribune (" + stockTribune + ")");
    System.out.println("1.VIP       - Rp 1.500.000");
    System.out.println("2.Festival  - Rp 800.000");
    System.out.println("3.Tribune   - Rp 500.000");
    System.out.println("4.Matikan Mesin");
    System.out.print("Pilih menu (1-4): ");

    int pilihan = scanner.nextInt();

    if (pilihan == 4) {
        System.out.println("Mematikan mesin Kiosk... Sampai jumpa!");
        break;
    }

    if (pilihan < 1 || pilihan > 4) {
        System.out.println("Pilihan tidak valid. Silakan coba lagi.");
        continue;
    }
```

Pembahasan:

Sistem dibungkus dalam perulangan tak terbatas while (true). Hal ini berfungsi untuk menjaga mesin Kiosk agar selalu menyala dan kembali menampilkan "Menu Utama" setelah pelanggan selesai bertransaksi. Mesin hanya akan mati (keluar dari perulangan menggunakan break) jika pengguna memilih opsi rahasia nomor 4. Jika pengguna salah ketik angka di luar 1-4, continue akan me-reset *loop* ke awal.

4.3. Penyesuaian Kategori dan Pencegahan Overselling

```java
System.out.print(s: "Masukkan jumlah tiket yang ingin dibeli: ");
int jumlahTiket = scanner.nextInt();

int stokSaatIni = 0;
int hargaSaatIni = 0;
int minUsiaSaatIni = 0;
String namaKategori = "";

if (pilihan == 1) {
    stokSaatIni = stockVIP; hargaSaatIni = hargaVIP; minUsiaSaatIni = minUsiaVIP; namaKategori = "VIP";
} else if (pilihan == 2) {
    stokSaatIni = stockFestival; hargaSaatIni = hargaFestival; minUsiaSaatIni = minUsiaFestival; namaKategori = "Festival";
} else if (pilihan == 3) {
    stokSaatIni = stockTribune; hargaSaatIni = hargaTribune; minUsiaSaatIni = minUsiaTribune; namaKategori = "Tribune";
}

if (jumlahTiket > stokSaatIni) {
    System.out.println("\n[ERROR] Maaf, stok tidak mencukupi. Sisa stok " + namaKategori + " hanya " + stokSaatIni + ".");
    continue;
}
```

Pembahasan:

Sistem mencatat jumlah tiket yang diminta, lalu menggunakan blok if-else untuk menyetel variabel penampung sementara (stokSaatIni, hargaSaatIni, minUsiaSaatIni) sesuai kategori yang dipilih. Setelah itu, sistem mengecek kondisi if (jumlahTiket > stokSaatIni) untuk mencegah *overselling* (kelebihan penjualan). Jika tiket yang diminta melebihi stok, program langsung memotong alur dengan continue, menolak transaksi, dan kembali ke Menu Utama.

4.4. Loop Validasi Usia dan Batal Darurat

```java
int tiketBerhasil = 0;
int totalTagihan = 0;
boolean batalDarurat = false;

System.out.println(x: "\n--- Pengisian Data Usia ---");
for (int i = 1; i <= jumlahTiket; i++) {
    int usia = 0;

    while (true) {
        System.out.print("Masukkan usia untuk Tiket ke-" + i + " (ketik -1 untuk batal): ");
        usia = scanner.nextInt();

        if (usia == -1) {
            batalDarurat = true;
            break;
        }

        if (usia <= 0 || usia > 100) {
            System.out.println(x: "Usia tidak logis. Silakan masukkan kembali.");
        } else {
            break;
        }
    }

    if (batalDarurat) {
        System.out.println(x: "\n[BATAL DARURAT] Proses dihentikan paksa. Kembali ke Menu Utama.");
        break;
    }
```

Pembahasan:

Sistem menggunakan perulangan for untuk menanyakan usia tiket satu per satu. Di dalamnya terdapat *nested loop* while (true) untuk validasi:

- Kebal Input Tidak Logis: Jika diinput nilai <= 0 atau > 100, program akan menolak dan terus mengulang permintaan. Perulangan while baru akan dihentikan dengan break jika usia masuk akal.
- Batal Darurat: Jika diinput angka -1, status batalDarurat berubah menjadi true, lalu break memutus *loop* input, dan kondisi if (batalDarurat) memicu break kedua untuk menghentikan paksa sisa pesanan secara keseluruhan.

4.5 Verifikasi Syarat Usia dan Cetak Nota

```java
            if (batalDarurat) {
                System.out.println(x: "\n[BATAL DARURAT] Proses dihentikan paksa. Kembali ke Menu Utama.");
                break;
            }

            if (usia >= minUsiaSaatIni) {
                System.out.println("Verifikasi sukses untuk Tiket ke-" + i + ".");
                tiketBerhasil++;
                totalTagihan += hargaSaatIni;

                if (pilihan == 1) stockVIP--;
                else if (pilihan == 2) stockFestival--;
                else if (pilihan == 3) stockTribune--;

            } else {
                System.out.println("[GAGAL] Tiket ke-" + i + " ditolak. Syarat usia " + namaKategori + " minimal " + minUsiaSaatIni + " tahun.");
            }
        }

        if (!batalDarurat || tiketBerhasil > 0) {
            System.out.println(x: "\n--- NOTA AKHIR ---");
            System.out.println("Dari " + jumlahTiket + " tiket yang coba dibeli, hanya " + tiketBerhasil + " tiket " + namaKategori + " yang berhasil diproses");
            System.out.println("Total tagihan: Rp " + totalTagihan);
        }
    }
    scanner.close();
```

Pembahasan

Setelah usia tervalidasi logis, program mengecek apakah usia >= minUsiaSaatIni. Jika memenuhi syarat, tiketBerhasil bertambah, totalTagihan diakumulasi, dan stok terkait dikurangi secara permanen. Jika gagal, program hanya mencetak pesan penolakan khusus untuk tiket tersebut, lalu membiarkan proses berlanjut ke tiket berikutnya tanpa mengganggu pesanan secara keseluruhan. Di akhir, Nota dicetak untuk menampilkan hasil transaksi yang sukses diproses.

V. Kesimpulan

Penggunaan perulangan `while (true)` terbukti efektif untuk menjaga mesin Kiosk tetap beroperasi melayani pelanggan secara terus-menerus, serta mengunci pengguna agar tidak bisa melanjutkan proses jika memasukkan data usia yang tidak logis. Sementara itu, perulangan `for` memfasilitasi sistem untuk menanyakan data usia secara presisi satu per satu sesuai jumlah pesanan tiket.

Selain itu, kombinasi percabangan `if-else` dengan perintah *control flow* seperti `break` dan `continue` memberikan fleksibilitas tinggi pada alur program. `continue` sukses diimplementasikan untuk mencegah *overselling* dengan mengembalikan pengguna ke menu utama, sedangkan `break` berhasil digunakan untuk mengeksekusi fitur "Batal Darurat" dan mematikan mesin. Secara keseluruhan, program Kiosk ini telah berhasil memenuhi seluruh *requirements* atau spesifikasi tugas yang diminta dengan memanfaatkan struktur kontrol logika dasar Java secara optimal.

VI. Daftar Pustaka

- Deitel, P., & Deitel, H. (2017). *Java: How to Program, Early Objects* (11th ed.). Pearson Education.
- Liang, Y. D. (2019). *Introduction to Java Programming and Data Structures, Comprehensive Version* (12th ed.). Pearson.
- Oracle Corporation. (2023). *The Java™ Tutorials: Control Flow Statements*. Diakses dari https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html
- Tim Asisten Praktikum Ilmu Komputer. (2026). *Modul 3: JAVA BASICS II)*. Jakarta Selatan : Universitas Pertamina.